# Dossier technique de développement

Architecture, modèle de données, flux, sécurité et plan de réalisation de l'application Allo Vaho et de ses interfaces web.

| | | |
|---|---|---|
| Version<br>**1.3** | Date<br>**28 septembre 2026** | Durée cible<br>**16 semaines** |
| Mobile<br>**Flutter, Android et iOS** | Backend<br>**Laravel, PostgreSQL** | Équipe<br>**1 dev mobile, 1 dev web** |
| Paiements<br>**Nokash, Apple, Google** | Hébergement<br>**VPS au Cameroun** | Référence visuelle<br>**Brief design v1.0** |

# Sommaire


1 Contexte et périmètre 2

2 Acteurs et droits 2

3 Architecture globale 3

4 Choix techniques 4

5 Modules fonctionnels 5

6 Flux d'abonnement et intégration Camtel 6

7 Cycle de vie d'un abonnement 7

8 Synchronisation des contacts et annuaire 8

9 Règles de visibilité des numéros 9

10 Paiements 10

11 Authentification 11

12 Notifications et rappels de paiement 11

13 Modèle de données (PostgreSQL) 12

14 Contrat d'API (extrait) 14

15 Architecture de l'application Flutter 15

16 Exigences d'interface issues du brief design 16

17 Back-office et portail Camtel 18

18 Sécurité et conformité 18

19 Environnements et DevOps 19

20 Planning et répartition de l'équipe 19

21 Risques et parades 21

22 Décisions validées 21


Référence visuelle et rédactionnelle : Brief design Allo Vaho v1.0 (28 septembre 2026). Décisions du porteur de projet intégrées au 28 septembre 2026.

# 1 Contexte et périmètre

Allo Vaho est un annuaire numérique mobile qui permet de souscrire, pour soi ou pour un proche, un forfait Camtel donnant accès aux appels, SMS et à internet illimités, puis de retrouver facilement les autres abonnés pour les appeler gratuitement.

La plateforme ne transporte ni les appels ni les SMS : ceux-ci passent par le numéroteur et l'application de messagerie natifs du téléphone, sur le réseau Camtel. Le rôle d'Allo Vaho est d'**encaisser**, de **transmettre la demande d'activation à Camtel**, de **suivre le cycle de vie de l'abonnement** et de **mettre en relation** les abonnés via un annuaire respectueux de la vie privée.

> **Version 1.3.** Ce document intègre les décisions du porteur de projet du 28 septembre 2026 (section 22) et s'aligne sur le *Brief design Allo Vaho v1.0*, qui fait foi pour tout ce qui est visuel et rédactionnel (section 16).

## Dans le périmètre de la version 1

### Application mobile

- Inscription par OTP SMS, Google, Apple (Facebook reporté en V1.1)
- Catalogue de packs, souscription pour soi ou pour un tiers
- Paiement via Nokash : Orange Money, MTN Mobile Money, carte Visa
- Annuaire des abonnés, favoris, contacts du répertoire déjà inscrits
- Masquage du numéro et demandes d'accès
- Invitations par lien, espace Entreprise et prestataires géolocalisés, numéros utiles
- Rappels de paiement et notifications

### Web (Laravel)

- API REST versionnée consommée par l'application
- Back-office administrateur : packs, utilisateurs, groupes, entreprises, numéros utiles, campagnes mail/SMS/push, tableaux de bord
- Portail partenaire Camtel : file des demandes d'activation, confirmation ou rejet
- Espace web Entreprise : souscription des plans Entreprise payés par Nokash
- Intégrations : Nokash, achats intégrés Apple et Google, passerelle SMS, Firebase, e-mail

## Hors périmètre

Téléphonie VoIP intégrée, messagerie interne, gestion technique de la ligne côté Camtel (provisioning réseau), facturation opérateur. Ces sujets restent chez Camtel ; Allo Vaho reçoit seulement la confirmation du traitement.

# 2 Acteurs et droits

| Acteur | Canal | Ce qu'il peut faire |
|---|---|---|
| **Visiteur inscrit** jamais payé | App | Voir les packs, l'espace Entreprise, les numéros utiles, inviter des contacts. **Pas d'accès à la liste des abonnés.** |
| **Abonné** a payé au moins une fois | App | Tout ce qui précède + annuaire, contacts inscrits, favoris, demandes de visibilité, abonnement de tiers, gestion de ses abonnements. |

| Acteur | Canal | Ce qu'il peut faire |
|---|---|---|
| **Bénéficiaire** | App / SMS | Personne abonnée par un tiers. Reçoit un SMS avec lien d'installation ; son abonnement l'attend à l'inscription et lui ouvre l'accès à l'annuaire. |
| **Entreprise** | App | Fiche entreprise ou prestataire de services, localisée sur la carte, visible par tous les abonnés tant que l'abonnement Entreprise est actif. |
| **Administrateur** | Back-office | Packs, groupes, modération entreprises, numéros utiles, campagnes, remboursements, statistiques. Rôles fins : super-admin, support, marketing, finance. |
| **Agent Camtel** | Portail partenaire | Traiter dans le portail partenaire les demandes d'activation : confirmer (avec référence Camtel) ou rejeter (avec motif). |

**Règle d'accès à l'annuaire.** Le drapeau `has_ever_paid` passe à vrai au premier abonnement *activé par Camtel*, qu'il ait été payé par l'utilisateur lui-même ou offert par un tiers, et ne redescend jamais. Un abonné expiré garde donc la consultation de l'annuaire, conformément au cahier des charges ; seul l'appel gratuit dépend de l'abonnement Camtel en cours.

# 3 Architecture globale

L'architecture retenue est un **monolithe Laravel modulaire** exposant une API REST, complété par des workers asynchrones. Pour une équipe de deux développeurs, c'est le meilleur compromis : un seul déploiement backend, des frontières de modules nettes, et la possibilité d'extraire un module plus tard si la charge l'exige.

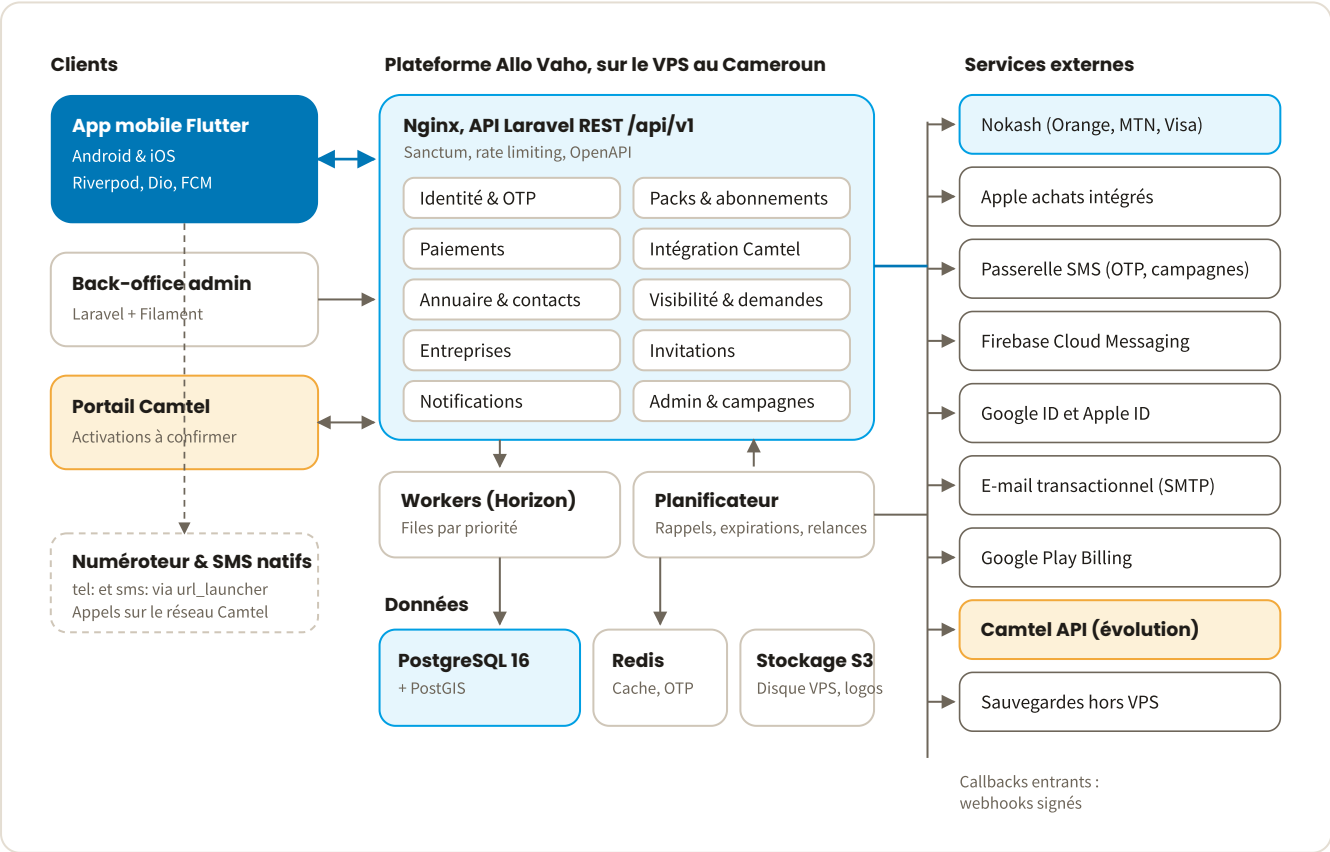



Figure 1. Vue d'ensemble. En bleu plein : le client principal ; en orange : tout ce qui touche à Camtel ; en pointillés : ce qui se passe hors de l'application. Camtel travaille dans le portail partenaire ; l'API Camtel reste une évolution possible.

### Principes d'architecture

- **API d'abord.** Le contrat OpenAPI est écrit et figé en début de sprint ; le dev mobile travaille sur un serveur de mock pendant que le dev web implémente. C'est ce qui permet aux deux ressources d'avancer en parallèle sans se bloquer.
- **Modules Laravel isolés** (`app/Domain/Subscriptions`, `app/Domain/Payments`…) avec leurs modèles, services, événements et politiques. Les modules communiquent par événements (`PaymentSucceeded`, `SubscriptionActivated`…).
- **Tout ce qui dépend d'un tiers est asynchrone** : paiement, envoi à Camtel, SMS, push. Une panne d'un prestataire ne bloque jamais l'application.
- **Adaptateurs interchangeables** pour paiements, SMS et Camtel : une interface, plusieurs drivers. Ajouter un second prestataire de paiement ou passer du portail Camtel à une API Camtel ne touche pas le métier.
- **Idempotence partout** sur les webhooks et les jobs : un callback reçu deux fois ne crée jamais deux activations.

## 4 Choix techniques

| Couche | Technologie | Justification |
|---|---|---|
| Mobile | Flutter 3.x (Dart), Riverpod, go_router, Dio, Freezed | Une seule base de code Android/iOS pour un seul dev mobile. Riverpod pour un état testable, Freezed pour des modèles immuables générés depuis l'OpenAPI. |
| Backend | Laravel (dernière version stable), PHP 8.3+ | Écosystème complet (queues, scheduler, notifications, Socialite, Sanctum) qui couvre 80 % du besoin sans code d'infrastructure. |
| Back-office | Filament 3 (dans le même projet Laravel) | Génère CRUD, tableaux, filtres et tableaux de bord en quelques heures par écran. Indispensable pour qu'un seul dev web tienne API + admin + portail Camtel. |
| Base de données | PostgreSQL 16 + PostGIS | Transactions fiables pour les paiements, JSONB pour les payloads de callback, PostGIS pour la recherche d'entreprises et d'abonnés à proximité. |
| Cache et files | Redis + Laravel Horizon | Files séparées par priorité, supervision des jobs, stockage OTP avec expiration native. |
| Authentification | Sanctum (jetons), Socialite, vérification des ID tokens Google/Apple | Jetons révocables par appareil, simples à gérer côté mobile. |
| Rôles admin | spatie/laravel-permission | Rôles et permissions fins, audit via spatie/laravel-activitylog. |
| Push | Firebase Cloud Messaging (APNs relayé) | Gratuit, un seul canal pour Android et iOS. Firebase étant hors du Cameroun, les messages ne contiennent aucune donnée personnelle : l'app récupère le détail auprès de l'API. |
| Infra | VPS existant au Cameroun, Docker Compose, Nginx, fichiers sur le disque du VPS (MinIO si le volume grossit) | Données hébergées au Cameroun sur l'infrastructure déjà disponible ; mêmes conteneurs du poste de dev à la production. |
| CI/CD | GitHub Actions (backend), Codemagic (Flutter) | Tests et déploiement automatiques, builds signés et envoi vers Play Console / TestFlight. |

| Couche | Technologie | Justification |
|---|---|---|
| Observabilité | GlitchTip (compatible Sentry, plus léger) sur le VPS, Laravel Pulse, logs avec rotation | Voir les erreurs de paiement et de callback avant les utilisateurs, sans faire sortir de données du pays ni surcharger le VPS. |

### Paquets Flutter clés

`flutter_contacts`, `permission_handler`, `phone_numbers_parser` (normalisation E.164), `firebase_messaging`, `google_sign_in`, `sign_in_with_apple`, `in_app_purchase` (StoreKit 2 et Google Play Billing), `url_launcher` (tel:, sms:), `share_plus`, `app_links` (liens d'invitation), `geolocator`, `flutter_secure_storage`, `drift` (cache local de l'annuaire), `webview_flutter` (page de paiement Nokash).

## 5 Modules fonctionnels

| Module | Fonctionnalités | Mobile | Web |
|---|---|---|---|
| Identité | Inscription OTP, Google, Apple (Facebook en V1.1) ; profil ; appareils ; suppression de compte | Écrans, SDK sociaux | OTP, vérif. tokens, Sanctum |
| Packs | Catalogue, détail, prix, durée, code d'offre Camtel | Catalogue | CRUD admin, API |
| Abonnements | Souscrire pour soi ou un tiers, renouveler, résilier le renouvellement, historique | Parcours, suivi du statut | Machine d'états, jobs |
| Paiements | Forfaits : Orange Money, MTN Mobile Money et Visa via Nokash. Plans Entreprise : achats intégrés Apple et Google dans les apps, Nokash sur l'espace web. Reçus ; remboursements validés par l'admin | Choix du moyen, attente, reçu, achats intégrés | Driver Nokash, vérification Apple et Google, webhooks, rapprochement |
| Camtel | Transmission des demandes, confirmation, rejet, SLA | Affichage du statut | Portail partenaire, alertes SLA |
| Annuaire | Liste des abonnés, recherche, contacts du répertoire inscrits, favoris, appel et SMS natifs | Sync contacts, cache local | Matching, API paginée |
| Visibilité | Numéro public/masqué, demandes d'accès, acceptation/refus | Réglages, boîte de demandes | Règles, politiques d'accès |
| Invitations | Lien d'invitation personnel, invitation d'un contact, suivi des conversions | Partage, deep links | Codes, attribution |
| Entreprises | Fiche entreprise ou prestataire de services, géolocalisation, abonnement de visibilité, recherche par catégorie, proximité et carte | Formulaire, carte, recherche | Modération, plans, espace web Entreprise |
| Numéros utiles | Pompiers, police, SAMU… par région, accessibles sans abonnement et hors ligne | Écran, cache | CRUD admin |
| Localisation | Position des entreprises et prestataires, facultative à la création, visible par tous les abonnés | Choix du point sur carte ou GPS | PostGIS, recherche de proximité |
| Notifications | Rappels de paiement, nouveaux contacts inscrits, campagnes admin | Centre de notifications | Scheduler, campagnes |
| Groupes | Groupes d'utilisateurs gérés par l'admin (ciblage, offres dédiées) | – | CRUD, ciblage |

# 6 Flux d'abonnement et intégration Camtel

C'est le flux critique de la plateforme. Il traverse cinq acteurs et contient une étape manuelle chez Camtel dont la durée n'est pas maîtrisée : l'application doit donc toujours afficher un statut clair et l'équipe doit être alertée quand une demande traîne.

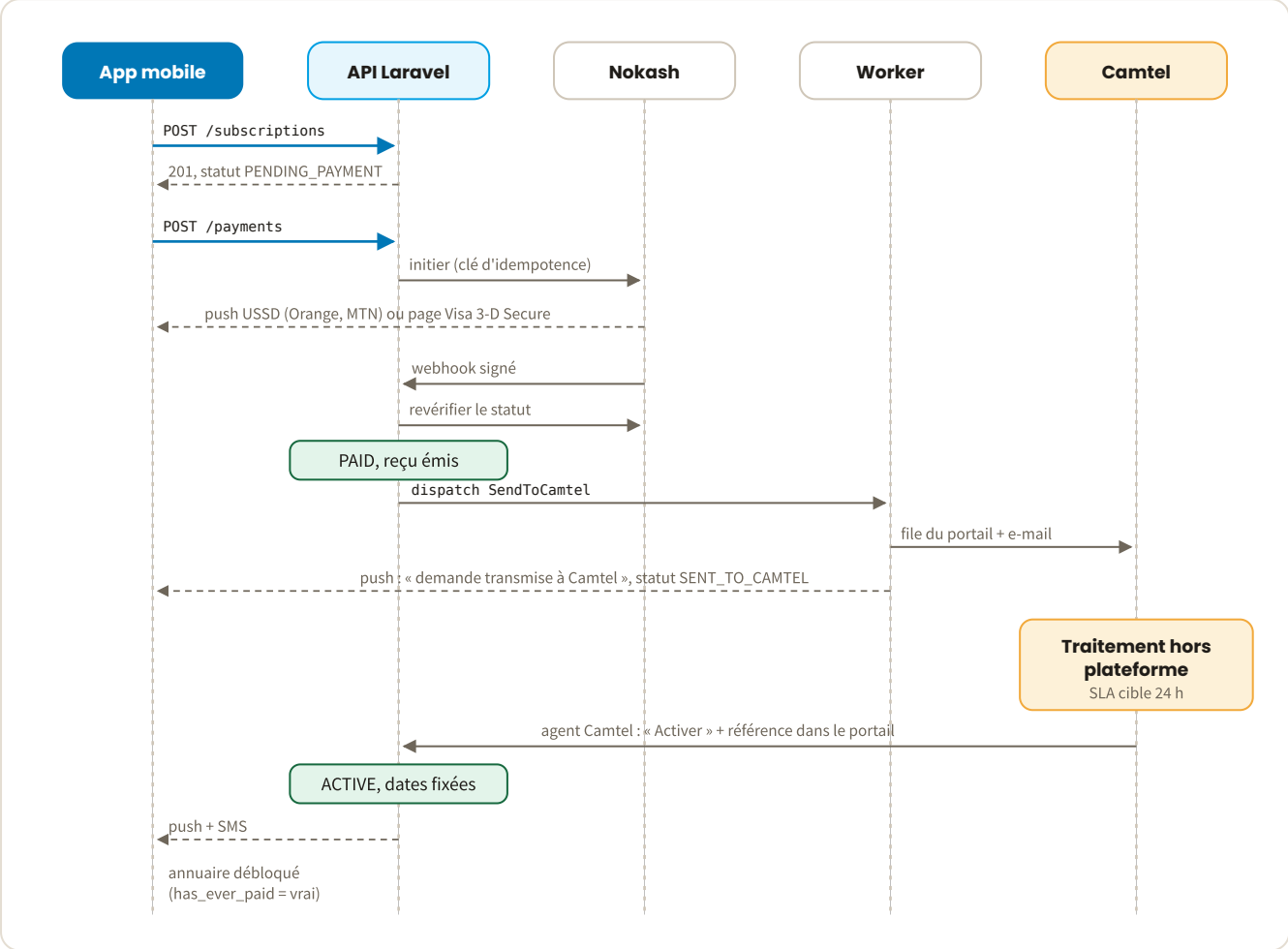



Figure 2. Séquence de souscription, du choix du pack à l'activation confirmée par Camtel.

## Intégration Camtel par portail partenaire

**Décision validée :** Camtel traite les demandes dans un portail partenaire fourni par Allo Vaho. Aucun développement n'est attendu côté Camtel. Le code passe malgré tout par une interface `CamtelGateway` dont le portail est l'implémentation V1 : une future API Camtel pourra s'y brancher sans toucher au métier.

### Côté Allo Vaho

Au passage en `PAID`, un job crée la demande dans `camtel_requests`, la place dans la file du portail et envoie un e-mail récapitulatif aux agents Camtel (sans données au-delà du nécessaire). Un résumé des demandes en attente part aussi chaque matin.

### Côté agents Camtel

Connexion nominative avec double authentification. File des demandes avec numéro bénéficiaire, pack, code d'offre Camtel et ancienneté. Action *Activer* (référence Camtel obligatoire) ou *Rejeter* (motif obligatoire), unitaire ou par lot, et export/import CSV pour un traitement groupé.

### Garde-fous

- **Contrôle du numéro bénéficiaire** avant paiement : format E.164 et préfixe Camtel (liste de préfixes fournie par Camtel, paramétrable).
- **SLA surveillé** : alerte admin à 12 h sans réponse, escalade à 24 h, notification à l'utilisateur pour le rassurer.
- **Rejet Camtel** : passage en `REJECTED` et notification avec le motif. La demande apparaît dans la file « Remboursements à valider » du back-office ; l'administrateur (rôle finance) valide, ce qui déclenche le remboursement Nokash vers le moyen de paiement d'origine (`REFUND_PENDING` puis `REFUNDED`). Aucun remboursement n'est automatique. Le délai maximal de traitement est un paramètre fixé par l'administrateur dans le back-office et affiché à l'utilisateur dans le message de rejet.
- **Traçabilité** : chaque demande conserve le payload envoyé, l'agent ou l'appel qui l'a traitée, et l'horodatage de chaque transition.

## 7 Cycle de vie d'un abonnement

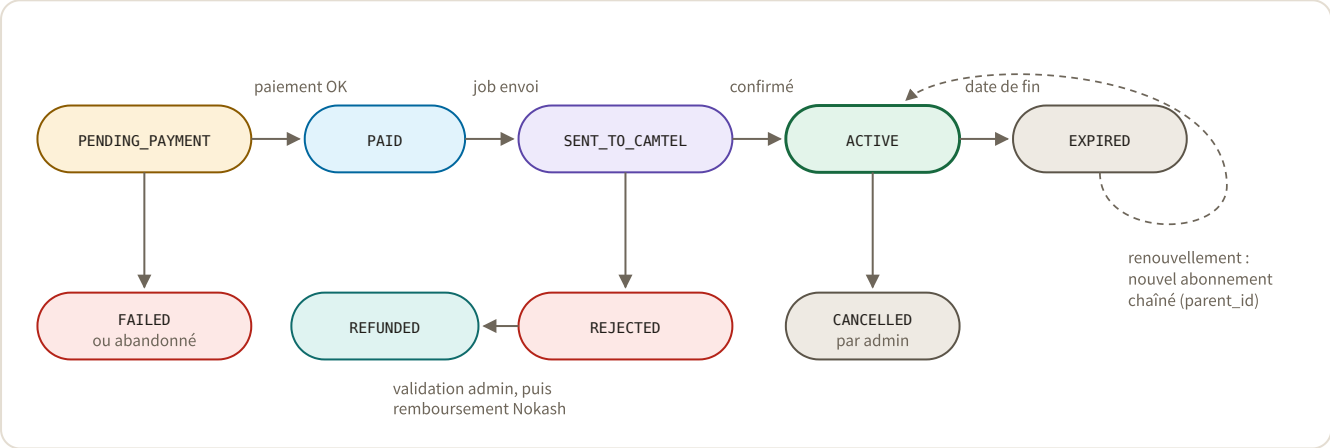



Figure 3. États d'un abonnement. Chaque transition est journalisée dans `subscription_events`. Couleurs des statuts reprises du brief design.

### Correspondance avec les statuts affichés

Le brief impose qu'un statut ne soit jamais porté par la couleur seule : icône, libellé et couleur vont toujours ensemble. L'API renvoie l'état technique ; l'application le traduit ainsi.

| État technique | Libellé affiché | Icône | Texte / fond (clair) |
|---|---|---|---|
| `PENDING_PAYMENT` | En attente de paiement | `schedule` | #8A5A00 / #FDF1D8 |
| `PAID` | Payé | `payments` | #00679E / #E1F3FC |
| `SENT_TO_CAMTEL` | Transmis à Camtel | `send` | #5B45A8 / #EEEAFB |
| `ACTIVE` | Actif | `check_circle` | #17693F / #E3F4EA |
| `REJECTED`, `REFUND_PENDING` | Rejeté + étape « Remboursement » dans la timeline | `cancel` | #B42318 / #FDE8E6 |

| État technique | Libellé affiché | Icône | Texte / fond (clair) |
|---|---|---|---|
| `REFUNDED` | Remboursé | `currency_exchange` | #0F6B6B / #DFF3F1 |
| `EXPIRED` | Expiré | `event_busy` | #5C5549 / #EEEAE3 |
| `CANCELLED` | Annulé (à ajouter au brief) | `block` | #5C5549 / #EEEAE3 |
| `FAILED` | Pas de badge : message « Le paiement n'a pas abouti. Aucun montant n'a été débité. » | – | – |

Premier principe du brief, « on sait toujours où on en est » : `GET /subscriptions/{id}` renvoie donc, en plus de l'état, `changed_at`, `next_step` et `deadline_at`. L'application peut ainsi afficher une phrase datée comme « Activation au plus tard demain à 14 h 03 » sans rien calculer elle-même.

- Un renouvellement ne réactive pas l'ancien enregistrement : il crée un nouvel abonnement lié par `parent_id`, qui suit le même parcours. Si le nouvel abonnement est activé avant la fin de l'ancien, sa date de début est calée sur la fin de l'ancien.
- **Résiliation par l'utilisateur** : elle désactive le renouvellement et les rappels ( `auto_renew_reminder = false` ) ; l'abonnement reste actif jusqu'à sa date de fin, le service étant déjà payé à Camtel.
- **Modification** (changement de pack) : traitée comme un nouvel abonnement démarrant à l'échéance du précédent, pour éviter les calculs de prorata avec Camtel.
- **Remboursement** : l'état intermédiaire `REFUND_PENDING` sépare la validation par l'administrateur de l'exécution chez Nokash ; les deux sont tracés dans `subscription_events`.
- Un paiement non finalisé après 30 minutes passe en abandonné ; l'utilisateur reçoit une relance une heure plus tard.

# 8 Synchronisation des contacts et annuaire

Objectif : comme sur Telegram, montrer à l'utilisateur quels contacts de son répertoire sont déjà sur Allo Vaho, et le prévenir quand un contact s'inscrit. Contrainte : ne jamais envoyer le carnet d'adresses en clair au serveur.

## Principe

1. L'application demande la permission Contacts après un écran explicatif (obligatoire pour Google Play et l'App Store).
2. Elle lit uniquement les numéros, les normalise en E.164 ( `+2376XXXXXXXX` ) et calcule `HMAC-SHA256(numéro, clé applicative)`. Les noms restent sur le téléphone.
3. Elle envoie les empreintes par lots de 500 à `POST /contacts/sync`. Seules les différences depuis la dernière synchronisation sont envoyées (ajouts, suppressions).
4. Le serveur stocke les empreintes dans `user_contacts` et renvoie la liste des utilisateurs correspondants avec, pour chacun, ce que le demandeur a le droit de voir.
5. L'application associe chaque résultat à son nom local et l'affiche dans l'onglet « Mes contacts sur Allo Vaho ».

### Notification « X a rejoint Allo Vaho »

À la vérification de l'OTP d'un nouvel utilisateur, l'événement `UserRegistered` déclenche un job qui cherche son empreinte dans `user_contacts`, renseigne `matched_user_id` et envoie un push à chaque propriétaire. Le push transporte l'identifiant de l'utilisateur ; l'application affiche le nom tel qu'enregistré dans le répertoire local quand il existe. Les envois sont groupés (un push par tranche de 15 minutes et par destinataire) pour éviter le spam lors d'une vague d'inscriptions.

> **Point de vigilance.** Les numéros camerounais formant un petit espace, une empreinte simple peut être inversée par force brute. On traite donc les empreintes comme des données personnelles : clé HMAC côté serveur en rotation contrôlée, limitation stricte du nombre de synchronisations par jour, pas d'export, purge des empreintes à la suppression du compte.

### Annuaire

- Accessible uniquement si `has_ever_paid = true` ; sinon, écran d'invitation à souscrire avec aperçu du nombre de contacts déjà inscrits (le nombre seul, pas les identités) : c'est le meilleur levier de conversion.
- Pagination par curseur, recherche par nom, ville, groupe. Cache local `drift` pour une consultation hors ligne.
- Un appui sur un abonné ouvre le numéroteur (`tel:`) ou l'app SMS (`sms:`) ; aucun appel ne passe par Allo Vaho.
- Favoris : ajout, retrait, tri manuel ; synchronisés sur le serveur pour survivre à un changement de téléphone.

## 9 Règles de visibilité des numéros

Par défaut, le numéro d'un abonné est public. Il peut le masquer. Le cahier des charges ajoute une règle de réciprocité : un abonné qui masque son numéro ne voit plus les numéros des personnes absentes de son répertoire. Tout le calcul se fait côté serveur : l'API ne renvoie jamais un numéro que le demandeur n'a pas le droit de voir.

```
function peutVoirNumero(A, B):          // A consulte la fiche de B
    si A == B                           → vrai
    si non A.has_ever_paid              → faux   // annuaire verrouillé
    si demandeAcceptee(A → B)           → vrai   // autorisation explicite de B
    si B.numero_masque
        et A absent du répertoire de B → faux
    si A.numero_masque
        et B absent du répertoire de A → faux   // réciprocité
    sinon                               → vrai
```

« A est dans le répertoire de B » se lit dans `user_contacts` : une ligne dont le propriétaire est B et dont `matched_user_id` vaut A. La relation est précalculée et indexée ; la règle s'évalue en une requête pour toute une page de l'annuaire.

### Demande d'accès à un numéro masqué

1. A voit la fiche de B (composant « Numéro masqué » du brief) avec le numéro remplacé par `+237 6•• •• ••42` et un bouton « Demander le numéro ».
2. La demande (message facultatif) crée une ligne `visibility_requests` en `PENDING` et un push à B.

3. B accepte ou refuse depuis sa boîte de demandes. A est notifié dans les deux cas.
4. B peut révoquer une autorisation à tout moment. A ne peut pas renvoyer de demande à B pendant 7 jours après un refus (règle du brief design), et au plus 10 demandes par jour au total, pour éviter le harcèlement.

> **Règle validée** : la réciprocité du masquage est confirmée par le porteur du projet. Elle reste isolée dans une *Policy* Laravel unique, couverte par des tests sur chaque combinaison (public/masqué, dans le répertoire ou non, demande acceptée ou non).

# 10 Paiements

Deux familles d'achats, deux logiques. Les **forfaits Camtel** sont un service consommé hors de l'application : ils se paient par **Nokash** (Orange Money, MTN Mobile Money, Visa) sur Android comme sur iOS. Les **plans Entreprise** achètent de la visibilité dans l'application : ils se vendent par **achat intégré** (Apple sur iOS, Google Play sur Android) et par **Nokash sur l'espace web Entreprise**. Le code s'organise autour de l'interface `PaymentGateway` (`initiate`, `verify`, `handleWebhook`, `refund`) avec trois implémentations : `NokashGateway`, `AppleIapGateway` et `GooglePlayGateway`.

| Achat | Canal | Moyen | Parcours et vérification |
|---|---|---|---|
| Forfait Camtel (pour soi ou un tiers) | App Android et iOS | Nokash | Choix Orange, MTN ou Visa ; push USSD à valider ou page Visa 3-D Secure en webview ; webhook signé puis revérification serveur. |
| Plan Entreprise | App iOS | Apple (StoreKit 2) | Abonnement auto-renouvelable ; transaction vérifiée par l'App Store Server API ; renouvellements, échecs et annulations reçus par les notifications serveur d'Apple. |
| Plan Entreprise | App Android | Google Play Billing | Abonnement Google Play ; jeton d'achat vérifié par la Google Play Developer API puis acquitté sous 3 jours (sinon Google rembourse) ; événements reçus par les notifications temps réel (Pub/Sub). |
| Plan Entreprise | Espace web Entreprise | Nokash | Souscription sur une page web ; l'app reconnaît ensuite le plan actif, quel que soit l'appareil. |

## Plans Entreprise gérés par l'administrateur

L'administrateur crée autant de plans qu'il le souhaite dans le back-office (nom, description, durée, avantages, prix Nokash, cible entreprise ou prestataire). Les stores imposent une contrainte : un plan ne peut être vendu dans une app que s'il existe aussi comme produit dans App Store Connect et dans la Google Play Console. Chaque plan porte donc `apple_product_id` et `google_product_id` ; tant qu'un identifiant est vide, le plan n'est proposé que sur l'espace web pour la plateforme concernée. Le back-office affiche clairement cet état pour éviter un plan « invisible » sans explication.

## Règles de robustesse

- Montants en entiers XAF, jamais en flottants ; les achats Apple et Google sont enregistrés dans leur devise d'origine avec le montant net reversé.
- Chaque paiement porte une clé d'idempotence ; un webhook déjà traité est acquitté sans effet.
- Aucun reçu ni webhook n'est cru sur parole : signature vérifiée, puis statut revérifié par appel serveur à serveur (Nokash, Apple, Google).

- Un même plan ne peut pas être actif deux fois pour une même fiche : si une entreprise paie par un canal alors qu'elle a déjà un plan actif par un autre, le nouvel achat prolonge la période au lieu de la dupliquer.
- Job de rapprochement toutes les 10 minutes sur les paiements `PENDING` de plus de 5 minutes (webhooks perdus).
- Remboursements Nokash uniquement après validation d'un administrateur (rôle finance). Les remboursements Apple et Google sont décidés par les stores : la plateforme les reçoit par notification et coupe le plan en conséquence.
- Rapport de réconciliation quotidien par canal (Nokash Orange, MTN, Visa ; Apple ; Google) dans le back-office, exportable pour la comptabilité.

**À savoir côté stores.** Apple et Google prélèvent une commission sur les achats intégrés (généralement 15 % pour les petites entreprises inscrites à leur programme dédié, 30 % sinon). Le prix dans les apps peut donc être plus élevé que sur l'espace web, mais les règles des deux stores interdisent d'indiquer dans l'app qu'un prix moins cher existe ailleurs, ou d'y placer un lien vers l'espace web. Nokash ne peut pas être proposé dans les apps pour les plans Entreprise : il reste réservé à l'espace web et aux forfaits Camtel.

## 11 Authentification

Le numéro de téléphone est l'identifiant pivot : sans numéro vérifié, pas d'annuaire ni d'abonnement. Les connexions Google et Apple accélèrent l'inscription mais exigent ensuite la vérification du numéro par OTP.

### OTP SMS

- Code à 6 chiffres, validité 5 minutes, stocké haché dans Redis.
- 3 essais par code, 5 envois par numéro et par heure, limites par IP et par appareil.
- Remplissage automatique : SMS Retriever API sur Android, `oneTimeCode` sur iOS.

### Connexions sociales

- Le SDK natif renvoie un ID token ; le serveur vérifie signature, audience et expiration.
- Liaison dans `social_accounts` ; un compte peut lier plusieurs fournisseurs.
- Sign in with Apple proposé sur iOS aux côtés de Google.
- Facebook Login reporté en V1.1 : il s'ajoute comme un fournisseur de plus, sans changement du modèle.

Jetons Sanctum par appareil, révocables depuis l'écran « Appareils connectés ». Stockage dans `flutter_secure_storage`. Suppression de compte disponible dans l'application (exigence des deux stores).

## 12 Notifications et rappels de paiement

| Déclencheur | Canal | Destinataires |
|---|---|---|
| J-7 avant échéance | Push | Payeur et bénéficiaire |
| J-3 avant échéance | Push + in-app | Payeur et bénéficiaire |
| J-1 et jour J | Push + SMS | Payeur et bénéficiaire |

| Déclencheur | Canal | Destinataires |
|---|---|---|
| J+1 (expiré) | Push + SMS | Payeur |
| Paiement abandonné | Push, 1 h après | Payeur |
| Statut Camtel (transmis, activé, rejeté) | Push + SMS pour l'activation | Payeur et bénéficiaire |
| Contact inscrit, demande de numéro, réponse | Push | Utilisateur concerné |
| Campagne admin | Mail, SMS, push, in-app | Tous, un groupe ou un segment |

Les textes suivent le ton du brief : vouvoiement, phrases de 12 mots maximum, montant et délai en chiffres, jamais « transaction » ni « requête ». Exemples de référence :

- Activation : « Forfait actif ! Appels, SMS et internet illimités jusqu'au 28 octobre. »
- Rappel J-3 : « Votre forfait se termine dans 3 jours. Renouvelez en 1 minute. »
- Contact inscrit : « Christelle Mbarga est sur Allo Vaho. Dites-lui bonjour. »

Les rappels sont calculés chaque matin à 8 h (heure de Douala) par le planificateur et mis en file ; les délais J-7/J-3/J-1 sont paramétrables dans le back-office. Chaque utilisateur peut couper les campagnes marketing mais pas les rappels liés à son abonnement. Toutes les notifications envoyées sont conservées dans le centre de notifications de l'application.

## 13 Modèle de données (PostgreSQL)

Clés primaires en `bigint` et identifiants publics en ULID pour l'API. Horodatages en `timestamptz` (UTC en base, affichage en heure de Douala). Suppression logique sur les tables utilisateur.

| Table | Colonnes principales | Remarques |
|---|---|---|
| `users` | id, ulid, phone_e164 (unique), phone_hash, name, email, avatar_path, number_hidden (bool), has_ever_paid (bool), city, status, referred_by | Index unique sur phone_e164 et phone_hash |
| `social_accounts` | user_id, provider (google, apple, facebook), provider_uid, email | Unique (provider, provider_uid) |
| `devices` | user_id, platform, fcm_token, app_version, last_seen_at | Un jeton Sanctum par appareil |
| `packs` | id, name, description, price_xaf, duration_days, camtel_offer_code, features (jsonb), group_id (null), is_active, sort | group_id pour une offre réservée à un groupe |
| `subscriptions` | id, payer_id, beneficiary_user_id (null), beneficiary_phone, pack_id, status, parent_id, starts_at, ends_at, auto_renew_reminder, camtel_ref | Statuts dont REFUND_PENDING ; index (status), (ends_at), (beneficiary_phone) |
| `subscription_events` | subscription_id, from_status, to_status, actor_type, actor_id, payload (jsonb), created_at | Journal immuable des transitions |
| `camtel_requests` | id, subscription_id, mode (portal, api), status, payload, sent_at, processed_at, processed_by, camtel_ref, rejection_reason, attempts | File traitée par le portail ou l'API |

| Table | Colonnes principales | Remarques |
|---|---|---|
| `payments` | id, payable_type, payable_id, user_id, provider, method, amount_xaf, status, provider_ref, idempotency_key (unique), raw_response (jsonb), paid_at, refunded_at | Polymorphe : abonnement ou entreprise ; provider = nokash, apple_iap ou google_play ; method = orange, mtn, visa ou iap |
| `user_contacts` | owner_id, phone_hash, matched_user_id (null), synced_at | PK (owner_id, phone_hash) ; index sur phone_hash et matched_user_id |
| `favorites` | user_id, favorite_user_id, position | PK composite |
| `visibility_requests` | id, requester_id, target_id, status (pending, accepted, refused, revoked), message, decided_at | Unique partiel sur (requester_id, target_id) en pending |
| `invitations` | id, inviter_id, code, phone_hash (null), channel, clicked_at, accepted_user_id | Lien du type allovaho.cm/i/{code} |
| `companies` | id, owner_id, kind (entreprise, prestataire), name, category_id, description, phone, email, website, address, location (geography), logo_path, status (draft, pending, published, suspended), visible_until | Localisation visible par tous les abonnés ; index GiST sur location |
| `company_plans` , `company_subscriptions` | plan : name, description, target (entreprise, prestataire, tous), duration_days, price_xaf (Nokash), features (jsonb), apple_product_id, google_product_id, is_active, sort ; abonnement : company_id, plan_id, payment_id, channel (nokash, apple_iap, google_play), store_transaction_id, starts_at, ends_at, auto_renewing, status | Plusieurs plans créés par l'admin ; même logique de rappels que les forfaits |
| `categories` | id, name, icon, parent_id | Catégories d'entreprises |
| `useful_numbers` | id, label, number, category, region, sort, is_active | Lisible sans authentification |
| `groups` , `group_user` | id, name, description ; group_id, user_id, added_by | Gérés par l'admin |
| `campaigns` | id, title, body, channels (jsonb), target_type, target_id, scheduled_at, status, stats (jsonb), created_by | Envoi par lots via les files |
| `notifications` | Table native Laravel | Centre de notifications in-app |
| `refunds` | id, payment_id, reason, requested_by, approved_by, approved_at, provider_ref, status | Validation admin obligatoire |
| `settings` , `activity_log` | Paramètres globaux (dont délai de remboursement et délais de rappel) ; audit admin | – |

**Abonnement pour un tiers non inscrit.** `beneficiary_user_id` reste vide et `beneficiary_phone` est renseigné. À l'inscription de ce numéro, un job rattache automatiquement l'abonnement à son nouveau compte et passe `has_ever_paid` à vrai ; le bénéficiaire a déjà reçu un SMS avec le lien d'installation dès l'activation.

# 14 Contrat d'API (extrait)

REST JSON, préfixe `/api/v1`, pagination par curseur, erreurs au format RFC 7807 (un `detail` rédigé pour l'utilisateur selon le ton du brief, et un `code` technique que l'app affiche en petit et permet de copier), documentation OpenAPI générée (Scribe) et publiée à chaque déploiement de recette.

| Méthode | Route | Rôle |
|---|---|---|
| POST | `/auth/otp/request` | Envoyer un code OTP |
| POST | `/auth/otp/verify` | Vérifier le code, créer ou connecter le compte |
| POST | `/auth/social/{provider}` | Connexion Google, Apple par ID token (Facebook en V1.1) |
| GET | `/me` | Profil, statut d'abonnement, accès annuaire |
| PATCH | `/me` | Nom, photo, masquage du numéro, localisation |
| DELETE | `/me` | Supprimer le compte |
| GET | `/packs` | Catalogue |
| POST | `/subscriptions` | Créer une souscription (pour soi ou un tiers) |
| GET | `/subscriptions` , `/subscriptions/{id}` | Historique, statut détaillé |
| POST | `/subscriptions/{id}/renew` | Renouveler |
| POST | `/subscriptions/{id}/cancel` | Résilier le renouvellement |
| POST | `/payments` | Initier un paiement |
| GET | `/payments/{id}` | Statut (écran d'attente) |
| GET | `/payments/{id}/receipt` | Reçu (toujours disponible après paiement) |
| POST | `/iap/apple/transactions` | Vérifier un achat intégré Apple |
| POST | `/iap/google/purchases` | Vérifier et acquitter un achat Google Play |
| POST | `/webhooks/apple` | Notifications serveur App Store |
| POST | `/webhooks/google` | Notifications temps réel Google Play |
| GET | `/company-plans` | Plans Entreprise disponibles pour la plateforme de l'appareil |
| POST | `/contacts/sync` | Envoyer les empreintes (delta) |
| GET | `/directory` | Annuaire paginé par 30 et filtrable |
| GET | `/directory/my-contacts` | Mes contacts inscrits |
| GET | `/favorites` ; POST / DELETE `/favorites/{user}` | Favoris |
| POST | `/visibility-requests` | Demander un numéro masqué |
| PATCH | `/visibility-requests/{id}` | Accepter, refuser, révoquer |
| POST | `/invitations` | Générer un lien d'invitation |

| Méthode | Route | Rôle |
|---|---|---|
| GET | `/companies` ; POST `/companies` | Rechercher, créer une fiche |
| GET | `/useful-numbers` | Numéros utiles (public) |
| GET | `/notifications` | Centre de notifications |
| POST | `/webhooks/payments/{provider}` | Callbacks Nokash (signés) |
| POST | `/webhooks/camtel` | Réservé à une future API Camtel |

## 15 Architecture de l'application Flutter

Organisation par fonctionnalité, en trois couches par fonctionnalité : présentation, domaine, données.

```
lib/
  core/            # thème, routeur, client Dio, intercepteurs, erreurs, i18n (fr, en)
  features/
    auth/          # data/ domain/ presentation/
    packs/
    subscriptions/
    payments/
    directory/     # annuaire, cache drift
    contacts/      # lecture, normalisation, hachage, sync delta
    visibility/
    favorites/
    invitations/
    companies/
    useful_numbers/
    notifications/
    profile/
  shared/          # widgets, formatage XAF et numéros
```

- **Navigation** : 5 onglets (Accueil, Annuaire, Entreprises, Numéros utiles, Profil). Les onglets restent visibles pour un visiteur ; l'annuaire affiche alors l'écran de souscription.
- **Hors ligne** : annuaire, favoris et numéros utiles en cache ; les numéros d'urgence sont aussi embarqués dans l'application.
- **Deep links** : invitations, retour de paiement Nokash, ouverture d'une notification.
- **Plans Entreprise** : liste chargée depuis l'API, croisée avec les produits réellement disponibles dans le store de l'appareil ; achat par `in_app_purchase`, puis envoi de la transaction au serveur qui seul active le plan. Bouton « Restaurer mes achats » obligatoire sur iOS.
- **Carte** : les entreprises et prestataires se positionnent sur une carte à la création de leur fiche ; les abonnés les retrouvent par proximité.
- **Synchronisation des contacts** dans un isolate pour ne pas figer l'interface, relancée au retour au premier plan si plus de 24 h se sont écoulées.
- **Qualité** : tests unitaires des règles, tests de widgets sur les parcours souscription et paiement, flavors dev, recette, prod.
- **Appareils visés** : Android 7+ et iOS 14+, avec une attention particulière aux téléphones d'entrée de gamme (taille d'APK, mémoire, réseau lent).

# 16 Exigences d'interface issues du brief design

Le *Brief design Allo Vaho v1.0* est le document de référence pour la marque, les couleurs, la typographie, le ton et les composants. Cette section traduit ses règles en exigences techniques vérifiables par les deux développeurs.

## Thème et jetons de design

Les couleurs vivent dans un seul fichier par plateforme, jamais en dur dans les écrans : `lib/theme/av_colors.dart` côté Flutter, variables CSS `--av-*` côté Laravel (back-office Filament, portail Camtel, espace web Entreprise).

| Jeton | Valeur | Rôle |
|---|---|---|
| `brand500` | #009FE3 | Logo, icônes, focus, grands aplats ; bouton principal en mode sombre |
| `brand600` | #0077B6 | `ColorScheme.primary` en clair : boutons, liens, texte bleu |
| `ink900` | #1F1C18 | Texte principal ( `onSurface` ) |
| `sand50` | #FAF7F2 | Fond de l'application ( `surface` ) |
| `clay600` | #6E665A | Texte secondaire, gris le plus clair autorisé |
| `sun` | #F2A93B | Accent favoris et « Le plus choisi », jamais en texte |
| `darkSurface` | #171512 | Fond sombre ; bouton #009FE3 avec texte #0B1A24 |

Mode clair par défaut (lisibilité en plein soleil). Typographies embarquées dans l'application : Nunito 700 à 900 pour titres, montants et boutons ; Noto Sans 400 à 600 pour le texte, avec chiffres tabulaires pour montants et numéros. Icônes en police variable Material Symbols Rounded, jamais en images.

## Formatage partagé

- Un utilitaire unique formate montants, numéros et heures avec espaces insécables : « 5 000 FCFA », « +237 677 12 34 56 », « 14 h 03 ». Il est utilisé par l'app, les SMS, les e-mails et les reçus.
- Aucun bouton à largeur fixe (le français est plus long que l'anglais) ; interfaces testées avec un texte système agrandi à 130 %.

## Bibliothèque de composants

Les composants du brief sont codés une seule fois dans `lib/shared/ui` et présentés dans un catalogue Widgetbook, pour les revues avec le porteur du projet : bouton (hauteur 52, libellé conservé pendant le chargement), champ numéro camerounais (+237 fixe, saisie groupée, erreur explicite), OTP à 6 cases avec lecture automatique du SMS, carte de forfait en forme de bulle (rayon 24/24/24/8), badge et carte de statut, timeline d'abonnement, ligne d'annuaire (hauteur 72, boutons SMS et Appeler de 48 dp), numéro masqué, sélecteur de paiement, feuille modale de récapitulatif, toasts (jamais pour une information de paiement).

## Payer sans avoir peur

Avant chaque paiement, un seul écran récapitule montant exact, bénéficiaire avec numéro complet, moyen et numéro débité. Le bouton répète le montant (« Payer 5 000 FCFA »). Après paiement, un reçu est toujours disponible, y compris hors ligne grâce au cache local.

## États obligatoires de chaque écran

- **Chargement** : squelettes à la forme du contenu, cache d'abord ; après 8 secondes, message « C'est plus long que d'habitude… » et bouton Réessayer.
- **Vide** : icône dans une bulle, une phrase, une action.
- **Erreur** : ce qui s'est passé, Réessayer, code technique copiable, données déjà affichées conservées.
- **Hors ligne** : bandeau avec la date du cache ; appel et SMS natifs toujours possibles ; paiement et demande de numéro désactivés avec explication.

## Budgets de performance

| Budget | Cible | Contrôle |
|---|---|---|
| Taille APK/AAB | ⩽ 25 Mo | Vérifiée à chaque build dans la CI, build en échec au-delà |
| Premier affichage | ⩽ 2 s en 3G | Mesuré sur Tecno Spark et Itel A70 à chaque fin de sprint |
| Polices embarquées | ≈ 600 Ko | Sous-ensembles latins uniquement |
| Pagination de l'annuaire | 30 contacts par page | Paramètre par défaut de l'API |
| Avatars | WebP 96 px, ⩽ 8 Ko | Conversion automatique par le serveur à l'envoi |
| Illustrations | SVG ⩽ 15 Ko, aucune photo décorative | Revue de code |
| Animations | 150 à 200 ms | Constantes de durée partagées |

## Accessibilité

Contraste AA partout ; libellés TalkBack et VoiceOver explicites (« Appeler Christelle Mbarga ») ; numéros lus chiffre par chiffre grâce aux `Semantics` Flutter ; statut, montant et échéance en 18 sp minimum ; focus visible de 2 px #009FE3 sur les interfaces web.

## Logo et supports

- Fichiers SVG et PDF du logo, dont une version monochrome vectorielle, à livrer avant la semaine 2 : c'est une dépendance du sprint 0.
- Tailles d'usage : splash 120 dp sur #009FE3, connexion 56 dp, barre d'accueil 28 dp, reçus et e-mails 32 px, web 40 px, symbole seul pour l'icône d'app et le favicon.
- Les QR codes et liens courts des supports imprimés sont tracés par canal : chaque support reçoit un code d'invitation dédié ( `invitations.channel` ) pour mesurer les inscriptions qu'il génère.
- Logos Orange Money, MTN MoMo, Visa et badges des stores à obtenir avec leurs chartes avant le sprint 2.

## 17 Back-office et portail Camtel

### Back-office administrateur

- Tableau de bord : inscriptions, conversions, chiffre d'affaires par moyen de paiement, demandes Camtel en attente.
- Utilisateurs : fiche, abonnements, paiements, suspension.
- Packs, plans Entreprise multiples (avec identifiants produit App Store et Google Play), numéros utiles, catégories.
- Paramètres : délai de remboursement affiché aux utilisateurs, délais des rappels, SLA Camtel.
- Groupes : création, ajout manuel ou par import CSV.
- Entreprises : modération avant publication.
- Campagnes : composer, cibler, programmer, suivre les envois.
- Paiements : réconciliation Nokash, Apple et Google, file des remboursements à valider (rôle finance).
- Espace web Entreprise : souscription et paiement Nokash du plan Entreprise.

### Portail partenaire Camtel

- Espace séparé, comptes nominatifs Camtel avec double authentification.
- File des demandes filtrable par date, pack, statut, avec ancienneté visible.
- Actions Activer (référence obligatoire) et Rejeter (motif obligatoire), unitaires ou par lot.
- Export CSV et import CSV de confirmations pour un traitement par lots.
- Aucun accès aux données non nécessaires (pas d'annuaire, pas de paiements).

## 18 Sécurité et conformité

- **Données personnelles** : l'application traite numéros, carnets d'adresses et localisations. L'hébergement au Cameroun est acté. Il reste à vérifier avec un juriste les autres obligations de la loi camerounaise sur la protection des données à caractère personnel (déclaration, consentement, durée de conservation) avant la mise en production.
- **Flux sortant du pays** : seuls Firebase (push), Apple et Google (authentification, achats) reçoivent des données, réduites au strict minimum ; aucune donnée d'annuaire ni de contacts ne quitte les serveurs camerounais.
- **Consentements** explicites et séparés pour les contacts, la localisation et les communications marketing ; politique de confidentialité publiée et liée dans les stores.
- **Transport et stockage** : TLS 1.2+ partout, chiffrement des colonnes sensibles (e-mail, localisation précise), sauvegardes chiffrées quotidiennes avec test de restauration mensuel.
- **API** : limitation de débit par route, protection contre l'énumération de numéros (réponses identiques qu'un numéro existe ou non), politiques Laravel sur chaque ressource.
- **Webhooks** : signature, liste d'IP autorisées quand le prestataire la fournit, horodatage anti-rejeu.
- **Back-office** : 2FA obligatoire, rôles minimaux, journal d'audit de toute action sensible.
- **Mobile** : référentiel OWASP MASVS niveau 1, obfuscation du code Dart, aucun secret dans l'application.
- **Test d'intrusion** externe recommandé avant l'ouverture publique.

## 19 Environnements et DevOps

| Environnement | Usage | Paiements | Camtel |
|---|---|---|---|
| Local | Développement (Docker Compose) | Simulateur interne | Simulateur |
| Recette | Démos de fin de sprint, tests métier | Sandbox Nokash, bac à sable Apple, testeurs de licence Google | Portail avec agents de test |
| Production | Public | Réel | Réel |

- Branches `main` (production) et `develop` (recette), revues de code croisées entre les deux développeurs sur les pull requests.
- Pipeline backend : lint (Pint, Larastan), tests Pest, build d'image, migration, déploiement sans interruption.
- Pipeline mobile : analyse, tests, build signé, distribution Firebase App Distribution pour la recette, puis Play Console et TestFlight.
- Hébergement : production sur le VPS existant (conteneurs Nginx, PHP-FPM, workers Horizon, PostgreSQL, Redis, GlitchTip). Configuration conseillée : 4 vCPU, 8 Go de RAM, 160 Go SSD minimum, à confirmer avec les caractéristiques actuelles du VPS. La recette tourne de préférence sur un second petit VPS, à défaut sur le même avec des conteneurs et bases isolés.
- Sauvegardes : dump PostgreSQL chiffré toutes les nuits et archivage WAL, copiés hors du VPS (second serveur ou stockage au Cameroun) ; une sauvegarde restée sur le VPS ne protège pas d'une panne du VPS. Restauration testée avant la mise en production.
- Supervision : GlitchTip, Horizon, ressources du VPS (CPU, mémoire, disque), alertes sur taux d'échec des paiements, files bloquées et demandes Camtel hors SLA.

## 20 Planning et répartition de l'équipe

Huit sprints de deux semaines. Le dev web porte le plus de charge (API, back-office, portail Camtel, intégrations) ; Filament et le contrat d'API figé en début de sprint sont les deux leviers qui rendent ce planning tenable. Le chef de projet prend en charge les dépendances externes : contrat Nokash, comptes des agents Camtel, produits dans App Store Connect et Google Play Console, profil de paiement des stores, comptes développeur Apple et Google, passerelle SMS.

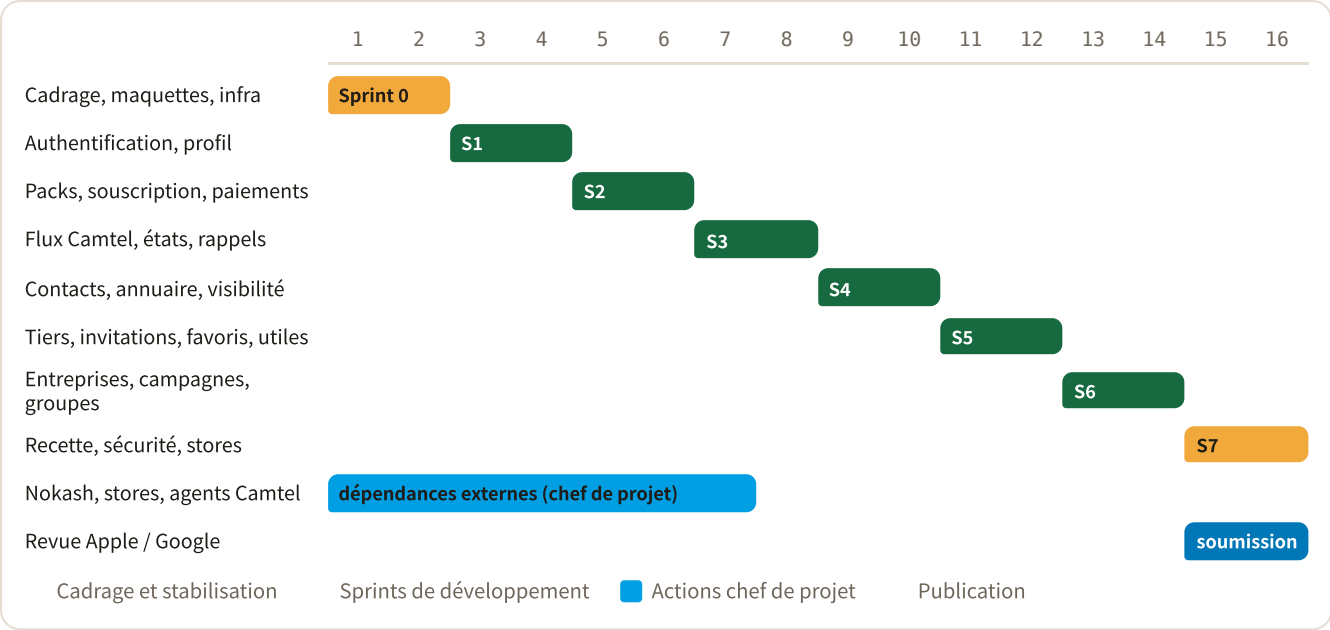



Figure 4. Planning sur 16 semaines. Le sprint 7 est volontairement court en développement : il sert à stabiliser.

| Sprint | Dev mobile (Flutter) | Dev web (Laravel) | Livrable démontrable |
|---|---|---|---|
| **0** S1–S2 | Squelette de l'app, thème et composants de base du brief design, polices, routeur, flavors, CI Codemagic | Projet Laravel, Docker, CI, thème Filament aux couleurs du brief, schéma de base v1, contrat OpenAPI v1, serveur de mock | App vide installable, API de santé en recette |
| **1** S3–S4 | Écrans OTP, Google, Apple, profil, appareils | OTP, passerelle SMS, vérification des tokens, Sanctum, profils | Inscription réelle de bout en bout |
| **2** S5–S6 | Catalogue, parcours de souscription, choix du paiement, écran d'attente, webview carte | Packs (admin Filament), abonnements, driver Nokash (Orange, MTN, Visa) en sandbox, webhooks | Paiement sandbox réussi |
| **3** S7–S8 | Suivi de statut, historique, centre de notifications, FCM | Machine d'états, portail Camtel, jobs, rappels, SLA, notifications, file des remboursements | Souscription payée puis activée par un agent de test |
| **4** S9–S10 | Permission et sync des contacts, annuaire, cache, masquage, demandes d'accès | Matching des empreintes, annuaire, politique de visibilité, notification « contact inscrit » | Deux téléphones de test qui se découvrent |
| **5** S11–S12 | Abonner un tiers, invitations et deep links, favoris, numéros utiles | Rattachement bénéficiaire, invitations, favoris, numéros utiles | Abonnement offert à un non-inscrit |
| **6** S13–S14 | Espace Entreprise et prestataires (création, carte, recherche), achats intégrés Apple et Google, restauration des achats | Entreprises, plans multiples, espace web Entreprise, vérification Apple et Google, webhooks stores, modération, groupes, campagnes | Plan Entreprise acheté sur iOS, Android et web |
| **7** S15–S16 | Corrections, performance sur appareils d'entrée de gamme, fiches stores | Corrections, test de charge, audit sécurité, mise en production | Version candidate soumise aux stores |

### Rituels

Point quotidien de 15 minutes, planification et démo en fin de chaque sprint avec le porteur du projet, rétrospective courte. Un tableau Kanban unique (colonnes : à faire, en cours, en revue, en recette, terminé) avec des tickets étiquetés mobile, web ou externe. Définition de « terminé » : code revu, testé, déployé en recette et validé en démo.

## 21 Risques et parades

| Risque | Impact | Parade |
|---|---|---|
| Lenteur de traitement des agents Camtel dans le portail | élevé | Formation des agents avant l'ouverture, e-mails de notification, SLA suivi avec escalade, traitement par lot et CSV. |
| Dépendance à un prestataire de paiement unique (Nokash) | élevé | Contrat et sandbox lancés en sprint 0, simulateur interne, supervision du taux d'échec ; l'interface de paiement permet d'ajouter un second prestataire si besoin. |
| Surcharge du dev web | élevé | Filament pour l'admin, contrat d'API figé, campagnes et entreprises placées en fin de planning (réductibles en V1.1). |
| Refus en revue App Store ou Google Play (contacts, achats) | moyen | Écrans de consentement soignés, Sign in with Apple, achats intégrés pour les plans Entreprise, aucun lien ni prix externe dans les apps, restauration des achats, soumission en test dès le sprint 5. |
| Fraude et coût des OTP SMS | moyen | Limites de débit, détection d'appareils suspects, plafond de dépense quotidien chez la passerelle SMS. |
| Données personnelles (carnets d'adresses) | moyen | Empreintes au lieu des numéros, consentement explicite, validation juridique avant production. |
| VPS unique (panne, saturation) | élevé | Supervision des ressources, sauvegardes hors VPS, procédure de reconstruction en conteneurs documentée et testée, montée en gamme du VPS ou second serveur dès que la charge l'exige. |
| Charge du sprint 6 (trois canaux de vente) | moyen | Campagnes et groupes décalables en V1.1 si les achats intégrés prennent plus de temps que prévu. |
| Réseau lent et appareils modestes | moyen | Pagination, cache local, images compressées, tests sur un appareil d'entrée de gamme à chaque sprint. |

## 22 Décisions validées

| Sujet | Décision | Effet dans ce document |
|---|---|---|
| Réciprocité du masquage | Confirmée | Règle de la section 9 validée |
| Échange avec Camtel | Portail partenaire | Sections 6 et 17 ; l'API Camtel devient une évolution |
| Rejet par Camtel | Remboursement validé par l'administrateur, délai fixé par l'administrateur | État REFUND_PENDING, table `refunds`, paramètre de délai dans le back-office |
| Paiement des forfaits | Nokash pour Orange, MTN et Visa | Section 10 |

| Sujet | Décision | Effet dans ce document |
|---|---|---|
| Vente des plans Entreprise | Achats intégrés Apple et Google, et Nokash | Apple sur iOS, Google Play sur Android, Nokash sur l'espace web |
| Plans Entreprise | Plusieurs plans créés par l'administrateur | Table `company_plans` avec identifiants produit des stores |
| Bénéficiaire abonné par un tiers | Accès à l'annuaire | `has_ever_paid` passe à vrai à l'activation |
| Facebook Login | Reporté | Prévu en V1.1 |
| Localisation | Visible par tous les abonnés ; sert à situer entreprises et prestataires | Localisation portée par les fiches, recherche sur carte |
| Hébergement | VPS existant, au Cameroun | Docker Compose sur le VPS, sauvegardes hors VPS |

## Derniers éléments à fournir pendant le sprint 0

1. **Caractéristiques du VPS** (processeur, mémoire, disque, système) et emplacement de la copie des sauvegardes.
2. **Comptes développeur** Apple et Google au nom de la société, avec profils de paiement renseignés, et inscription aux programmes de commission réduite.
3. **Premiers plans Entreprise** (noms, durées, prix Nokash et prix stores) pour créer les produits dans les deux consoles.